Module 9 Week B — Integration Task: NL → Cypher Pipeline

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

This is the Week B capstone. You will compose **NL question** → **intent detection** → **slot extraction** → **parameterized Cypher** → **Neo4j execution** → **results** into a single command-line tool.

The integration is the engineering primitive **schema-bounded NL→code generation**. The core deliverable implements that primitive deterministically — a templated mapper over a finite, enumerable space of question shapes against a bounded recipe-KG schema. Challenge Tier 3 implements the same primitive with a live LLM (`GraphCypherQAChain`) for cases where the input distribution is open and templating no longer scales. Both arms are first-class implementations of the same engineering pattern; the choice between them is a design decision driven by the shape of the input distribution and the safety properties the deployment requires.

---

1. Setup

If you haven't already installed the Module 9 packages, go back to the Setup: install before the concept session section of the reading and complete the install before proceeding.

> ***Windows users.*** *Commands below assume Bash/Zsh (macOS/Linux/WSL2/Git Bash). On PowerShell or CMD, activate your venv with* `.venv\Scripts\Activate.ps1` *(PowerShell) or* `.venv\Scripts\activate.bat` *(CMD), and confirm Docker Desktop is running with the* ***WSL2 backend*** *before* `docker compose up -d`*. The Tier 3* `requirements-challenge.txt` *install (Ollama) is verified on the WSL2 path, not native PowerShell — install and run Ollama from inside WSL2. Full cross-platform command tables are in the Pre-Work install plan.*

> ***Submission flow — all Module 9 Week B assignments.*** *Week B uses a fork-and-submit flow (drill, lab, integration, both stretches) instead of GitHub Classroom. Steps 1–3 below are the new part; steps 4+ are the same workflow you used in the lab.*

1. **Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m9-i9b and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave "Copy the main branch only" checked.
2. **Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork.)
3. **Clone *your* fork (not the upstream repo):**

   ```
   git clone https://github.com/<your-github-username>/m9-i9b.git
   cd m9-i9b
   ```

4. **Create your working branch:**

   ```
   git checkout -b integration-9b-nl-cypher
   ```

5. **Bring Neo4j up locally and install deps** (the install is cached from the lab — should be fast):

   ```
   docker compose up -d
   pip install -r requirements.txt
   ```

   Wait for the Neo4j container to print `Started.` in `docker compose logs neo4j` before continuing.

6. **Load the fixture:**

   ```
   python load_fixture.py
   ```

   This loads the same ~200-node recipe graph the lab used and asserts the node + relationship counts. If the assertion fails, the graph did not load cleanly — bring the container down (`docker compose down`) and try again.

7. **Quick sanity check.** Even before you write any code, the CLI is callable:

   ```
   python cli.py "Find Italian recipes"
   ```

   On the unmodified starter this fails meaningfully — `pipeline.answer` raises `NotImplementedError` with a pointer back to this guide. That's the expected starting state.

---

2. What you'll build

You are building the pipeline that turns a natural-language question into structured results from the recipe KG. The flow is:

```
NL question
    │
    ▼
mapper.intent.detect_shape(question)         — classify into one of 15 ShapeIds,
    │
    ▼
mapper.slots.extract_slots(question, shape) — pull entity slots out of the quest
    │
    ▼
mapper.compile.compile_to_cypher(shape, slots) — (cypher_string, params_dict)
    │
    ▼
driver.session().run(cypher, **params)       — parameterized execution over Bolt
    │
    ▼
list of result rows (dicts keyed by RETURN aliases)
```

`pipeline.answer(driver, question)` wires those four steps together. `cli.py` is the user-facing entry point: parse argv, open a driver, route through `answer` , pretty-print rows.

The template repo also ships a **bundled reference linker** under `linker/` — a complete implementation of the W9B Lab's entity-linking contract, included as a reference implementation you can read alongside your own. The integration's deterministic mapper does **not** import `linker/` at all; the mapper operates on NL question shapes, not on linker output. Even if your lab linker fell short of the W9B Lab thresholds, you can still complete the integration in full. The mapper, `pipeline.py` , and `cli.py` are the integration's load-bearing surface.

---

3. The bounded 15-template surface (core deliverable)

The recipe-KG schema is bounded: **6 labels** (`:Recipe` , `:Cuisine` , `:Ingredient` , `:Author` , `:Technique` , `:Entity`) and **6 relationship types** (`:USES_INGREDIENT` , `:OF_CUISINE` , `:BY_AUTHOR` , `:REQUIRES_TECHNIQUE` , plus the two `:SUBCLASS_OF` hierarchies on cuisine and ingredient). A schema that small admits a finite, enumerable space of question shapes — the engineering pattern is **bounded schema + templated implementation = deterministic, deployable, grading-stable**.

The Cypher string the mapper emits is also a **white box**: each clause names a label, a relationship, or a property that exists in the schema, and a reader can trace every returned row back to the exact path that produced it. That is the explainability pay-off the Reading frames in §8.5 — when a downstream caller needs to justify *why* this row came back (not just *that* it did), the white-box Cypher path is what they cite. The same explainability holds for the Tier 3 live-LLM variant below, with one caveat: the LLM-emitted Cypher is white-box *after* it is emitted, but the emission process is the black box you trade off against the live-LLM's open-input flexibility.

These are the 15 NL question shapes the deterministic mapper supports. Anything outside this surface raises `UnsupportedQueryError` .

#	NL question	Cypher shape
1	"Find recipes that use ginger"	single-hop `:USES_INGREDIENT`
2	"Find recipes by author Maria Rossi"	single-hop `:BY_AUTHOR` filter on `name`
3	"Find Italian recipes"	direct `:OF_CUISINE` match on `Cuisine.name`
4	"Find Asian recipes"	`:OF_CUISINE` + `[:SUBCLASS_OF*0..]` hierarchical traversal
5	"Find Sichuan recipes that use ginger"	conjunction: `:OF_CUISINE` + `:USES_INGREDIENT`
6	"Find Chinese recipes that use ginger"	conjunction + `[:SUBCLASS_OF*0..]`
7	"Find recipes that require wok technique"	single-hop `:REQUIRES_TECHNIQUE`
8	"Find recipes by author Maria Rossi that use basil"	conjunction `:BY_AUTHOR` + `:USES_INGREDIENT`
9	"Find Italian recipes ranked by popularity"	`ORDER BY r.popularityScore DESC LIMIT 10`
10	"Find recipes with prep time under 30 minutes"	property filter `r.prepMinutes < $max_minutes`
11	"Find ingredients used in Italian recipes"	inverse: `:OF_CUISINE` then `:USES_INGREDIENT`
12	"Find authors of Sichuan recipes"	`:BY_AUTHOR` + `:OF_CUISINE` + `[:SUBCLASS_OF*0..]`
13	"Find recipes that use peppercorn or any subtype"	ingredient hierarchy `[:SUBCLASS_OF*0..]`
14	"Find recipes that use ginger but not garlic"	negation via `WHERE NOT EXISTS { ... }`
15	"Find recipes optionally tagged with `<technique>` "	`OPTIONAL MATCH` on technique; `$technique` slot is still required

The 15 canonical Cypher templates are in `mapper/shapes.py` under `CANONICAL_CYPHER` — read them. They are course-provided and you should treat them as the gold the autograder grades against; your job is **classification + slot extraction + lookup**, not template authoring. Every template parameterizes its slot values through `$param` syntax — slot values flow through the params dict only, never through string formatting.

The hierarchical traversal `[:SUBCLASS_OF*0..]` is load-bearing for shapes 4, 6, 12, and 13. The cuisine taxonomy is 3–4 levels deep (`Sichuan → Chinese → Asian → World`); the `*0..` quantifier means "zero or more hops," so a query for "Asian recipes" matches recipes whose cuisine **is** `Asian` (zero hops) **and** all descendants (1+ hops). Reversing the direction or omitting `*0..` will silently miss most of the answer set.

The gold (question_text, shape, slots, expected_row_count) triples for all 15 questions live in `data/eval_questions.jsonl` . The autograder runs your pipeline on each and checks the result set matches the canonical template's output.

---

4. Implementing `mapper/intent.py`

`detect_shape(question: str) -> ShapeId | None` classifies the question into one of the 15 `ShapeId` values, or returns `None` .

Approach. A small set of keyword / regex rules over the lowercased question text. The shape vocabulary is narrow:

- `"by author <name>"` / `"by <Name>"` → author shapes (q2, q8, q12)
- `"<cuisine name>"` → cuisine shapes (q3, q4, q5, q6, q9, q11, q12)
- `"use <ingredient>"` / `"with <ingredient>"` → ingredient shapes (q1, q5, q6, q8, q13)
- `"but not <ingredient>"` → q14 (negation)
- `"ranked by popularity"` / `"most popular"` → q9
- `"under <N> minutes"` → q10
- `"ingredients used in"` → q11 (inverse)
- `"authors of"` → q12
- `"or any subtype"` / `"or any kind"` → q13
- `"optionally tagged"` → q15
- `"require <technique>"` → q7

Apply rules in priority order — more-specific shapes before less-specific. q14 (`"but not"`) must fire before q1 (`"use ginger"`) — both questions contain `"use ginger"` , but q14 is the correct answer. q8 (`"by ... that use"`) must fire before q2 and q1 for the same reason.

Worked detection. Question: `"Find Sichuan recipes that use ginger"` . Lowercased: `"find sichuan recipes that use ginger"` . The classifier sees `sichuan` (a cuisine name in the KG vocabulary) AND `use ginger` (ingredient pattern) AND nothing matching `"but not"` , `"by author"` , `"under N minutes"` , etc. The conjunction `cuisine + ingredient` without subtype-traversal language matches **q5** (`OF_CUISINE` direct + `USES_INGREDIENT`). Returning q6 would also produce the correct answer set on this specific question — `[:SUBCLASS_OF*0..]` collapses to the direct edge when no descendants exist — but the gold for the eval set is q5, so detection precision matters.

Prefer None over a false positive. A classifier that returns a wrong `ShapeId` on a supported question is a bug; a classifier that returns a confident `ShapeId` on an off-template question is the silent-failure mode. When in doubt, return `None` and let the orchestrator raise `UnsupportedQueryError` .

Bare ingredient names without a context cue are NOT q1. The cue contract above lists `"use <ingredient>"` and `"with <ingredient>"` — those are the patterns that map to ingredient shapes. A question that mentions an ingredient name without one of those cues (e.g., the adversarial `"find tomato (which is a fruit)"`) is off-template and must return `None` . A naive `any(ingredient_name in question for ingredient_name in KG_INGREDIENTS)` scan is the silent-failure pattern this rule is designed to catch.

---

5. Implementing `mapper/slots.py`

`extract_slots(question: str, shape: ShapeId) -> dict` extracts the slot values the shape's Cypher template needs.

Approach.

- **spaCy NER** for `PERSON` entities (q2, q8 author slot). Reuse the spaCy pipeline you installed for Module 6 — `nlp = spacy.load("en_core_web_sm")` , iterate `doc.ents` , pick the `PERSON` span.
- **Vocabulary match** for cuisines and ingredients. The lists are small (16 cuisines, 40 ingredients) — a literal case-insensitive substring match against a hand-authored vocabulary list is fine. The bundled reference linker exposes the canonical name lists; you can import them or copy them into `slots.py` .
- **Regex** for q10: `r"under (\d+)\s*minutes"` to pull the integer threshold.
- **Split-on-but-not** for q14: split the question on `"but not"` / `"without"` to get the positive and negative ingredient slots.

Canonical form matters. The KG stores `"Italian"` not `"italian"` , `"ginger"` not `"Ginger"` . Match against the schema vocabulary rather than echoing the surface form of the question, otherwise the Cypher template — which compares against `Cuisine.name` exactly — returns zero rows.

Slot keys must match the `$param` names in `shapes.CANONICAL_CYPHER` . Inspect `mapper/shapes.py` to see what each template expects. Returning a slot dict missing a required key surfaces as a Neo4j `ParameterMissing` error at query time — that is fail-loud and desired.

Example return values:

```
extract_slots("Find recipes that use ginger", ShapeId.Q1)
# -> {"ingredient": "ginger"}

extract_slots("Find Sichuan recipes that use ginger", ShapeId.Q5)
# -> {"cuisine": "Sichuan", "ingredient": "ginger"}

extract_slots("Find recipes with prep time under 30 minutes", ShapeId.Q10)
# -> {"max_minutes": 30}

extract_slots("Find recipes that use ginger but not garlic", ShapeId.Q14)
# -> {"ingredient": "ginger", "exclude_ingredient": "garlic"}
```

---

6. Implementing `mapper/compile.py`

`compile_to_cypher(shape: ShapeId, slots: dict) -> tuple[str, dict]` returns the Cypher template string and the params dict that will bind into it.

This function is small. Look up the static template from `CANONICAL_CYPHER[shape]` . Return that string unchanged (it already has the `$param` placeholders). Return the slots dict unchanged (or a shallow copy) as the params bound by the driver.

Worked example. `compile_to_cypher(ShapeId.Q4, {"cuisine": "Asian"})` returns:

```
MATCH (r:Recipe)-[:OF_CUISINE]->(c:Cuisine)
MATCH (c)-[:SUBCLASS_OF*0..]->(:Cuisine {name: $cuisine})
RETURN r.name AS recipe
ORDER BY r.name
LIMIT 50
```

with `{"cuisine": "Asian"}` as the params dict. The Neo4j driver binds `$cuisine` at execution time, never at string-build time.

Always parameterized; never f-string interpolation. The autograder includes an AST-level inspection of this file's source code: it walks the AST and confirms there is no f-string that interpolates a slot value into a string passed to `session.run` . This is the load-bearing safety beat from the Reading — the query body is a static template, identical across all calls; slot values flow through `$params` only.

```
# CORRECT — template-from-dict pattern
template = CANONICAL_CYPHER[shape]
return template, dict(slots)

# WRONG — f-string interpolation of slot values into the Cypher body
return f"MATCH (r:Recipe) WHERE r.name = '{slots['name']}'", {}
```

The wrong pattern is what allows Cypher injection: a slot value `"; DROP DATABASE neo4j; //"` becomes executable Cypher.

---

7. The fail-loud surface — `UnsupportedQueryError`

When `detect_shape()` returns `None` , the pipeline does not silently return an empty list. It raises `UnsupportedQueryError` (from `mapper/errors.py`), and the error message names every one of the 15 supported shape templates so the failure is explicit and the next engineering step (extending the mapper for a 16th shape) is discoverable from the error alone.

```
from mapper import detect_shape, UnsupportedQueryError

def answer(driver, question: str) -> list[dict]:
    shape = detect_shape(question)
    if shape is None:
        raise UnsupportedQueryError(question)
    # ... continue through extract_slots → compile_to_cypher → session.run
```

Do **not** convert `None` from `detect_shape` into an empty result list. An empty list is what a valid, supported question with no matching rows looks like — collapsing the off-template case into the same return shape destroys the diagnostic signal a learner (or downstream caller) needs.

The Reading walks through the contrast between a deterministic mapper's correct output and a live LLM's typical failure mode on a multi-hop conjunction question — see Reading §8 (Text2Cypher — schema-bounded NL→code generation). Both the integration's deterministic mapper (this page's core deliverable) and Tier 3's `GraphCypherQAChain` (below) are first-class implementations of that primitive.

---

8. Orchestration — `pipeline.py` and `cli.py`

`pipeline.answer(driver, question)` is the orchestrator:

```
from mapper import detect_shape, extract_slots, compile_to_cypher, UnsupportedQue

def answer(driver, question: str) -> list[dict]:
    shape = detect_shape(question)
    if shape is None:
        raise UnsupportedQueryError(question)
    slots = extract_slots(question, shape)
    cypher, params = compile_to_cypher(shape, slots)
    with driver.session() as session:
        result = session.run(cypher, **params)
        return [row.data() for row in result]
```

`cli.py` is the user-facing entry point:

```
python cli.py "Find Italian recipes"
```

Required behavior:

- Read the question from `argv[1]` . If missing, print a usage hint to stderr and exit with code `2` .
- Build a Neo4j driver from the `NEO4J_URI` / `NEO4J_USER` / `NEO4J_PASSWORD` environment variables (defaults in `cli.py` point at the local docker-compose Neo4j).
- Call `answer(driver, question)` and pretty-print each row on its own line. If the result list is empty, print `(no results)` .
- Catch `UnsupportedQueryError` , print its message to stderr, and exit with code `1` .
- Close the driver in a `finally` block.

Exit codes — `0` success, `1` unsupported question, `2` usage / unexpected error — keep the CLI scriptable; a caller can branch on the exit code without parsing stderr.

---

9. Tests

```
pytest tests/ -v
```

`tests/test_mapper.py` is LLM-free and runs against the Neo4j service container in CI. It covers:

- **Intent detection** — each of the 15 canonical eval questions classifies to its gold `ShapeId` ; a handful of adversarial off-template questions return `None` .
- **Slot extraction** — extracted slot dicts match the gold slots in `data/eval_questions.jsonl` for each question.
- **Compile output** — `compile_to_cypher` returns the canonical template string and the slot dict unchanged.
- **Parameterization (AST check)** — walks `mapper/compile.py` and confirms no f-string interpolation flows into a string passed to `session.run` . The check looks for an `ast.JoinedStr` (f-string) whose value reaches a `session.run` call.
- **Result-set correctness** — runs the full pipeline against the Neo4j fixture and asserts the result set for each of the 15 canonical questions matches the canonical Cypher template's output.
- **Fail-loud** — feeds 3 off-template questions through `answer` and asserts each raises `UnsupportedQueryError` with a message that names all 15 supported shapes.

The autograder workflow (`.github/workflows/m9-i9b-autograder.yml`) brings up a Neo4j service container, loads the fixture, and runs `pytest tests/ -v --tb=short` . The CI run never installs `requirements-challenge.txt` and never calls a live LLM.

---

10. Challenge Extensions

These extensions are opt-in. None affect your base integration grade. They are graded as Honors signals when present and substantive. Pick whichever tier matches the depth you want to go to — you do not need to do them in order, and you do not need to do all of them. When you submit a tier, name it explicitly in your PR description (e.g., "Tier 1 extension below the base submission").

Challenge Tier 1 — Five additional Cypher templates

Add the 16th–20th question shapes to the deterministic mapper. Extend `mapper/shapes.py` with five new `ShapeId` entries, each backed by a `(intent, slot-extraction rule, Cypher template)` triple following the existing pattern in shapes 1–15. At least one of the five must be a **multi-hop conjunction** that is not already covered by shapes 1–15 (e.g., "Italian recipes by author X that use technique Y" — the conjunction across cuisine + author + technique). For each new shape: author one positive `eval_questions.jsonl` entry, run it through the full pipeline, and include the executed Cypher in your PR description. Your existing 15-shape tests must still pass — extending the space is the deliverable, not refactoring it.

Challenge Tier 2 — Learned intent classifier

Replace the rule-based intent classifier in `mapper/intent.py` with a 15-class model trained on synthetic NL questions. Two acceptable routes:

- **scikit-learn route.** A TF-IDF + `LogisticRegression` or `LinearSVC` pipeline, trained on ~200 synthetic questions you author (or generate templatically) per shape, with held-out evaluation.
- **distilbert route.** Fine-tune `distilbert-base-uncased` as a 15-class head on the same ~200-per-shape synthetic set.

Measure classification accuracy on a held-out 30-question dev split. The trained classifier replaces `classify_intent(question)` — the rest of the pipeline is unchanged. Compare the accuracy to the rule-based baseline (`run_dev_eval.py --intent-only --rule-based`). Report both numbers and discuss where the learned classifier wins and where the rule-based baseline still wins. Pin model artifacts and the training script under `challenge/intent/` .

Challenge Tier 3 — Live-LLM `GraphCypherQAChain` swap-in

Tier 3 is the live-LLM implementation of the same schema-bounded NL→code generation primitive — what production teams build when the input distribution is open enough that templating no longer covers the space. The deterministic mapper above and Tier 3's `GraphCypherQAChain` are both first-class implementations of the primitive; the choice between them is a design decision.

Tier 3 has four implementation surfaces: the **schema preamble + few-shots** (course-provided), the **query-shape allowlist** (course-provided; the load-bearing Text2Cypher safety surface), the **LLM client resolution** (`get_llm_client()`), and the **chain orchestration** (`build_prompt` + `run_chain`).

10.3.1 Setup

Install the Tier 3 dependencies (separate from `requirements.txt`):

```
pip install -r requirements-challenge.txt
```

That brings in `langchain` , `langchain-neo4j` , `langchain-openai` , `langchain-anthropic` , and `ollama` (the hosted-API packages are present so the OPENAI_API_KEY / ANTHROPIC_API_KEY paths below resolve without an extra install).

Install the Ollama runtime, then pull a model:

```
ollama pull phi3:mini-4k-instruct-q4_K_M    # ~2.4 GB
# or
ollama pull qwen2.5:1.5b-instruct-q4_K_M
```

Hardware floor. Local Ollama needs **≥16 GB RAM** to run alongside Neo4j and an editor without swap. On a lighter machine the system will swap aggressively, page faults will dominate, and per-token latency goes from 100 ms to multi-second.

If your hardware is below the floor, you have two paths:

- **Hosted-API path.** `get_llm_client()` resolves to `ChatOpenAI` or `ChatAnthropic` if `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` is set in your `.env` . An instructor-distributed key may be available — ask in the cohort channel. Per the Compute Rule, no Tier 3 deliverable **requires** paid API access; the hosted-API path is optional.
- **Skip Tier 3.** Tier 3 is a Challenge Extension, not a base requirement. Skipping it does not affect your base grade for the integration.

> ***Production context.*** *In production, teams running their own LLMs typically use* ***vLLM*** *— a GPU-based inference server (the production analog of Ollama). Vendors like Red Hat AI publish vLLM-ready quantized versions of Llama, Granite, Mistral, and Qwen. Not required for the integration.*

10.3.2 The schema preamble and few-shots

The LLM gets the bounded schema as a system-prompt preamble, plus a small set of input-output exemplars. Both live in `challenge/few_shots.py` and are course-provided.

```
# challenge/few_shots.py — SCHEMA_PREAMBLE (verbatim)
SCHEMA_PREAMBLE = """
This is a Neo4j property-graph schema. Use only these labels and
relationships.

Labels: Recipe, Cuisine, Ingredient, Author, Technique. Every node also
carries :Entity for uniqueness.

Relationships:
  (:Recipe)-[:USES_INGREDIENT]->(:Ingredient)
  (:Recipe)-[:OF_CUISINE]->(:Cuisine)
  (:Recipe)-[:BY_AUTHOR]->(:Author)
  (:Recipe)-[:REQUIRES_TECHNIQUE]->(:Technique)
  (:Cuisine)-[:SUBCLASS_OF]->(:Cuisine)
  (:Ingredient)-[:SUBCLASS_OF]->(:Ingredient)

Always use parameterized Cypher with $param syntax. Read-only clauses only
(MATCH, OPTIONAL MATCH, WHERE, RETURN, WITH, ORDER BY, LIMIT, UNION).

When a question names a cuisine or ingredient category that has subtypes,
include [:SUBCLASS_OF*0..] in the path so the answer covers descendants.
""".strip()
```

What each component contributes:

- **Labels and relationships enumeration** — constrains the LLM to the recipe-KG vocabulary. Without this, the LLM invents plausible-but-wrong labels (`:Dish` , `:Spice`) that don't exist in the graph and return zero rows.
- **"Always use parameterized Cypher with `$param` syntax"** — pushes the LLM toward the same safety beat the deterministic mapper enforces. The LLM does not always comply; that's why the allowlist (§10.3) is a defense-in-depth layer.
- **"Read-only clauses only"** — narrows the LLM's hypothesis space toward `MATCH` / `RETURN` patterns and away from `CREATE` / `SET` / `DELETE` . Again, the allowlist enforces this absolutely; the preamble is a soft prior.
- **`[:SUBCLASS_OF*0..]` instruction** — the hierarchical-traversal idiom is non-obvious from the schema alone. Naming it explicitly raises the rate at which the LLM produces shape-4-style queries on questions like "Find Asian recipes".

`EXAMPLE_PAIRS` carries three input-output exemplars: `"Find Asian recipes"` (shape 4, hierarchical), `"Find recipes that use ginger"` (shape 1, single-hop), and `"Find Sichuan recipes that use ginger"` (shape 5, conjunction). At least one demonstrates the `[:SUBCLASS_OF*0..]` idiom so the LLM has a worked precedent to imitate.

10.3.3 The query-shape allowlist

`challenge/allowlist.py` exposes `validate_query_shape(cypher: str) -> None` , which raises `UnsupportedCypherError` if the Cypher contains any clause in `FORBIDDEN_CLAUSES` :

```
FORBIDDEN_CLAUSES = {
    "CREATE", "MERGE", "DELETE", "DETACH", "SET", "REMOVE",
    "DROP", "CALL", "LOAD", "USING", "FOREACH",
}
```

The allowlist is course-provided. Read its body — it strips string literals and comments before tokenization (so `"DELETE"` inside a quoted name does not trigger), extracts uppercase keywords of length ≥3, and raises on the first match in `FORBIDDEN_CLAUSES` .

This is the load-bearing Text2Cypher safety surface. Neo4j Community has no per-user RBAC — there is no way to grant the Bolt session a role that excludes writes. The Python-side allowlist (combined with connection-string discipline — using a read-only `reader` user where one is configured) is what stands between LLM-emitted Cypher and the database. **Validate before you execute. Always.**

Adversarial walkthrough. The user types `"delete all recipes"` . The LLM, despite the read-only preamble, emits:

```
MATCH (r:Recipe) DELETE r
```

Without the allowlist, the driver executes the Cypher and wipes 120 recipes from the fixture. With the allowlist:

```
from challenge.allowlist import validate_query_shape, UnsupportedCypherError

cypher = "MATCH (r:Recipe) DELETE r"
try:
    validate_query_shape(cypher)
    # ... session.run only reached on success
except UnsupportedCypherError as e:
    # raised: "Forbidden clause 'DELETE' in LLM-generated Cypher; only read claus
    ...
```

`run_chain` (§10.5) catches `UnsupportedCypherError` and returns a dict with `rejected=True` and the rejection reason; the user sees an explicit refusal, the database is untouched, and the autograder summary distinguishes "LLM emitted unsafe Cypher" from "LLM emitted safe-but-wrong Cypher" (see §10.6).

The Tier 3 autograder includes 6 adversarial mutation tests — `"delete all recipes"` , `"drop the database"` , `"set every entity to NIL"` , `"create a backdoor admin"` , `"detach delete recipe 1"` , `"call apoc.export.cypher"` . Each must raise `UnsupportedCypherError` .

10.3.4 `get_llm_client()` — provider resolution

`challenge/llm_client.py` exposes `get_llm_client(model: str = "phi3:mini-4k-instruct-q4_K_M")` . The 5-step resolution order is course-provided dispatch logic; your TODO is the wrapper construction in steps 1–4.

Resolution order:

1. **`OLLAMA_HOST` env var** → `ChatOllama(model=model, base_url=OLLAMA_HOST)` . For learners running Ollama on a remote machine.
2. **Local Ollama at `http://localhost:11434`** → `ChatOllama(model=model)` . Before returning, the dispatcher runs `ollama list` and confirms `model` is present; if not, it raises `OllamaModelMissingError` with the exact pull command:

   ```
   Model 'phi3:mini-4k-instruct-q4_K_M' not pulled. Run: ollama pull phi3:mini-4
   ```

   The presence check fires **before** the client is returned so a missing pull surfaces as a clean error rather than an opaque connection error at `.invoke()` time.

3. **`OPENAI_API_KEY` in env** → `ChatOpenAI` (hosted path; optional per Compute Rule).
4. **`ANTHROPIC_API_KEY` in env** → `ChatAnthropic` (hosted path).
5. **Otherwise** → raise `NoLLMClientAvailableError` with installation guidance.

The dispatcher (steps 1, 2 presence-check, and 5) is course-provided. Your TODOs are the wrapper returns in steps 1–4 — instantiate the appropriate LangChain client and return it. The client must expose a `.invoke(prompt)` method (LangChain Runnable convention) so `chain.run_chain` can call it uniformly across providers.

10.3.5 `GraphCypherQAChain` — `build_prompt` and `run_chain`

`challenge/chain.py` exposes two functions:

`build_prompt(question: str) -> str` assembles `SCHEMA_PREAMBLE` + `EXAMPLE_PAIRS` + the user question into a single LLM prompt. The exact format is up to you, but at minimum: start with `SCHEMA_PREAMBLE` , append each `EXAMPLE_PAIRS` entry as `"Q: ...\nCypher: ..."` , and end with `"Q: {question}\nCypher:"` so the LLM continues by emitting Cypher.

`run_chain(driver, llm_client, question) -> dict` runs one question end-to-end:

1. Build the prompt via `build_prompt(question)` .
2. Invoke the LLM — `llm_client.invoke(prompt)` .
3. Parse the LLM response to extract a Cypher string and a params dict (the few-shot format is `"Cypher: ...\nParams: {...}"`).
4. Call `validate_query_shape(cypher)` . Catch `UnsupportedCypherError` and short-circuit to a rejection result.
5. If validation passed, run the Cypher via `session.run(cypher, **params)` and return the rows.

The return dict has keys `question` , `cypher` , `params` , `rows` , `rejected` (bool), and `rejection_reason` (str or None). This shape is what the Tier 3 tests assert on.

10.3.6 Tier 3 scoring methodology

The four-bullet methodology block below appears verbatim in two places: the module docstring at the top of `challenge/chain.py` and this Integration Guide section. Either is authoritative; they say the same thing.

> - *The 15 canonical eval questions in data/eval_questions.jsonl are scored by exact-result-set equivalence against the deterministic mapper's output on the same fixture graph (the deterministic mapper is the gold).*
> - *A Tier 3 answer is correct iff the executed Cypher returns exactly the same set of result rows as the deterministic mapper for that question; row order matters only for the two ranked questions (#9, #12) where ORDER BY is in the canonical shape.*
> - *A Tier 3 answer that raises UnsupportedCypherError (allowlist rejection) counts as incorrect for that question but is REPORTED SEPARATELY in the autograder summary so learners can distinguish "LLM emitted unsafe Cypher" from "LLM emitted safe-but-wrong Cypher".*
> - *Aggregation: report per-question correctness plus an overall accuracy (correct / 15). No partial credit on rows.*

This methodology answers the choices a learner could otherwise make differently and produce a different number: what counts as "correct" (exact result-set equivalence against the deterministic mapper); whether row order matters (only for q9 and q12); how rejections are reported (separately, not as silent wrong answers); and how the overall score aggregates (mean across 15 questions, no partial credit).

10.3.7 Running Tier 3

Locally with live Ollama:

```
# After requirements-challenge.txt is installed and the model is pulled
python -c "
from neo4j import GraphDatabase
from challenge.llm_client import get_llm_client
from challenge.chain import run_chain

driver = GraphDatabase.driver('bolt://localhost:7687', auth=('neo4j','testtest'))
llm = get_llm_client()
print(run_chain(driver, llm, 'Find Asian recipes'))
"
```

In CI, the Tier 3 autograder does NOT call a live LLM. It reads `challenge/mock_llm_cache.json` (a fixed map of `question → canonical Cypher response`) and substitutes the cached response in place of `llm_client.invoke()` . This keeps CI deterministic and free of network dependencies.

Tier 3 tests live at `tests/test_challenge_tier3.py` . They cover the build-prompt format, the allowlist rejection of all 6 adversarial Cypher mutations, and the result-set correctness across the 15 cached responses.

---

11. Common Pitfalls

- **f-string interpolation in `compile.py` .** Returning `f"MATCH (:Cuisine {{name: '{slots['cuisine']}'}}) ..."` rather than the parameterized template fails the AST check and exposes the database to injection. Always return the static template from `CANONICAL_CYPHER` and pass slot values through `$params` .
- **Forgetting `OPTIONAL MATCH` when the shape requires it.** q15 uses `OPTIONAL MATCH` so recipes **without** the named technique still appear in the result set with `technique = null` . A plain `MATCH` silently drops them. **Note:** q15 still extracts and binds a `$technique` slot value — the "optionally tagged" phrasing means "use `OPTIONAL MATCH` ," not "no technique value is needed." The concrete question in the eval set is `"Find recipes optionally tagged with wok technique"` ; `wok` is the slot value.
- **Reversing the `[:SUBCLASS_OF*0..]` direction.** The arrow on `:SUBCLASS_OF` points from the specific to the general — `(:Sichuan)-[:SUBCLASS_OF]->(:Chinese)-[:SUBCLASS_OF]->(:Asian)` . To get all cuisines under `:Asian` , traverse **from** `:Cuisine` **outwards** to the named parent: `MATCH (c)-[:SUBCLASS_OF*0..]->(:Cuisine {name: $cuisine})` . Reversing the arrow returns only the root.
- **(Tier 3) Ollama model not pulled.** `get_llm_client()` raises `OllamaModelMissingError` with the exact pull command. Run that command, not anything else.
- **(Tier 3) Validating the LLM's Cypher after executing it.** `validate_query_shape(cypher)` must be called **before** `session.run(cypher, ...)` . Executing first and validating after defeats the safety layer — the database write has already happened.
- **(Tier 3) Skipping the allowlist for trusted-looking responses.** Every LLM response is run through `validate_query_shape` . Trust is not a property of any single response; the allowlist is the contract.

---

12. Submit

1. Push your branch:

   ```
   git push origin integration-9b-nl-cypher
   ```

2. Open a PR **within your own fork** — base `main` , compare `integration-9b-nl-cypher` , **base repository = your fork** (GitHub defaults to upstream; change it).
3. Wait for the autograder workflow to run on your fork. Aim for green CI. If a check fails, read the test output, fix, push again — the workflow re-runs on each push.
4. Your PR description must include:
   - A short summary of which 15 shapes you covered and any you found difficult to classify.
   - A note on whether you attempted Tier 3, and if so which provider path (`Ollama` / `OPENAI_API_KEY` / `ANTHROPIC_API_KEY`) you used.
   - Paste your PR URL into TalentLMS → Module 9 Week B → Integration Task to submit this assignment.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.